

zeros() Function

zeros () Function is used to create 2D array with all zeros.

Syntax:-

```
numpy.zeros(shape, dtype=float, order='C')
```

Where,

shape – shape of new array. It can be an int which will represent number of elements or can be tuple of int. ex:- (3, 2)

rows

columns

dtype – The desired data-type for the array.

Order - Whether to store multi-dimensional data in row-major (C-style) or column-major (Fortran-style) order in memory. It can be C or F.

Creating 2D Array using zeros () Function

Syntax:-

```
from numpy import *
```

```
array_name = zeros(shape, dtype=float)
```

Ex:-

```
from numpy import *
```

```
a = zeros((3, 2))
```



```
a = array([ [0., 0.],  
            [0., 0.],  
            [0., 0.] ])
```

0.	0.	0.	0.	0.	0.
a[0][0]	a[0][1]	a[1][0]	a[1][1]	a[2][0]	a[2][1]

Accessing 2D Array Elements

*from numpy import **

`a = zeros((3, 2))`

`print(a[0][0])`

`print(a[0][1])`

`print(a[1][0])`

`print(a[1][1])`

`print(a[2][0])`

`print(a[2][1])`



<code>a = array([</code>	<code>[0., 0.],</code>
	<code>[0., 0.],</code>
	<code>[0., 0.]]</code>

	[0]	[1]
[0]	0.	0.
[1]	0.	0.
[2]	0.	0.

Accessing 2D Array using for loop

*from numpy import **

a = zeros((3, 2))

```
a = array([ [0., 0.],  
            [0., 0.],  
            [0., 0.] ])
```

Without index

```
for r in a:  
    for c in r:  
        print(c)  
    print( )
```

Outer for loop

Inner for loop

Inner for loop

With index

```
n = len(a)  
for i in range(n):  
    for j in range(len(a[i])):  
        print(a[i][j])  
    print( )
```

Outer for loop

The outer for loop represents the rows and the inner for loop represents the columns in each row.

Accessing 2D Array using while loop

```
from numpy import *
```

```
a = zeros((3, 2))
```

```
n = len(a)
```

```
i = 0
```

```
while i < n :
```

```
    j = 0
```

```
    while j < len(a[i]):
```

```
        print(a[i][j])
```

```
        j+=1
```

```
    i+=1
```



```
a = array([ [0., 0.],  
            [0., 0.],  
            [0., 0.] ])
```

ones() Function

ones () Function is used to create 2D array with several rows and columns with all 1s.

Syntax:-

```
numpy.ones(shape, dtype=float, order='C')
```

Where,

shape – shape of new array. It can be an int which will represent number of elements or can be tuple of int. ex:- (3, 2)

rows

columns

dtype – The desired data-type for the array. Default is float.

Order - Whether to store multi-dimensional data in row-major (C-style) or column-major (Fortran-style) order in memory. It can be C or F.

Creating 2D Array using ones () Function

Syntax:-

```
from numpy import *  
array_name = ones(shape, dtype=float)
```

Ex:-

```
from numpy import *  
a = ones((3, 2))
```



```
a = array([ [1., 1.],  
            [1., 1.],  
            [1., 1.] ])
```

1.	1.	1.	1.	1.	1.
a[0][0]	a[0][1]	a[1][0]	a[1][1]	a[2][0]	a[2][1]

Accessing 2D Array Elements

*from numpy import **

`a = ones((3, 2))`

`print(a[0][0])`

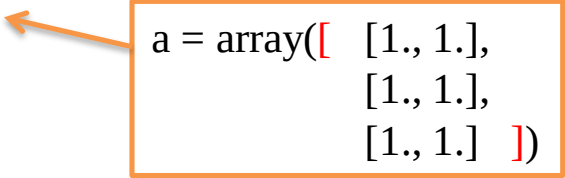
`print(a[0][1])`

`print(a[1][0])`

`print(a[1][1])`

`print(a[2][0])`

`print(a[2][1])`



```
a = array([ [1., 1.],  
           [1., 1.],  
           [1., 1.] ])
```

	[0]	[1]
[0]	1.	1.
[1]	1.	1.
[2]	1.	1.

Accessing 2D Array using for loop

*from numpy import **

a = ones((3, 2))

*a = array([[1., 1.],
[1., 1.],
[1., 1.]])*

Without index

```
for r in a:  
    for c in r:  
        print(c)  
    print( )
```

Outer for loop

Inner for loop

Inner for loop

With index

```
n = len(a)  
for i in range(n):  
    for j in range(len(a[i])):  
        print(a[i][j])  
    print( )
```

Outer for loop

The outer for loop represents the rows and the inner for loop represents the columns in each row.

Accessing 2D Array using while loop

```
from numpy import *
```

```
a = ones((3, 2))
```

```
n = len(a)
```

```
i = 0
```

```
while i < n :
```

```
    j = 0
```

```
    while j < len(a[i]):
```

```
        print(a[i][j])
```

```
        j+=1
```

```
    i+=1
```



```
a = array([ [1., 1.],  
           [1., 1.],  
           [1., 1.] ])
```